Software Design Specification

for

Pantry Inventory Management System

Version 1.0

Prepared by Andrew Feran, Arselle Conteh, Benjamin Bruinsma, Davi Borges, Nickolas Diaz

Florida Polytechnic University

06/07/2025

Table of Contents

1. Introduction
- 1.1 Purpose
- 1.2 System Overview

2. Design Considerations
- 2.1 Assumptions
- 2.2 General Constraints

3. System Architecture

4. Detailed System Design
- 4.1 Class Diagram
- 4.2 Activity Diagram
- 4.3 Sequence Diagram
- 4.4 State Transition Diagram
- 4.5 Algorithms for Components/Methods
- 4.6 Database Design
  - Users
  - Pantry Items
  - Notifications
  - Shopping Lists
  - Shopping List Items

5. User Interface Design

1. Introduction

1.1 Purpose

This document's purpose is to conglomerate design ideas between team members and provide a system structure to be improved on. The specifics contained in this document are detailed implementation foundations, design choices, diagrams, and interface designs.

1.2 System Overview

The project is our Pantry Inventory Management System that shall have the following capabilities:

- Manage pantry items (add, read, edit, delete)
- Give detailed information on items
- Routinely create shopping lists
- Receive reminders on expiration dates
- Use authentication to secure user data

2. Design Considerations

2.1 Assumptions

We assume that:

- There are no more than 500 items in the pantry at a time
- There are no more than 10 concurrent users
- The HTTPS protocol is secure
- The probability of collision of hashed passwords is zero due to cryptographic primitives
- Users may access this site from any number of web supported device types

This system will inherently rely on the security and reliability of:

- Python 3 as the codebase
- Flask as the backend of the web application
- MySQL as the database
- Apache2 as the web server
- Ubuntu as the host OS

2.2 General Constraints

The following are some general constraints on the solution along with their described impact.

- Environment: The software in its early stages will be run in an Ubuntu WSL on a Windows personal computer which will affect the uptime as well as the performance

- Public URL: In its early stages, the application will use ngrok to provide a URL which remote users can access to perform QA and functional testing
- Storage Limitations: The system shall not use redundancy or RAID configurations to ensure that there is storage available.
- Backup Data: The database shall take a transaction log every 15 minutes and a full backup every 24 hours- both of which shall be cleared after 7 days.
- Interoperability requirements: The system is web based, so we need to ensure the compatibility across all modern web browsers, such as Chrome, Mozilla, Safari, etc… With no layout breaking
- Performance requirements: Responsive to user input quickly, having a loading time that should be under 1 second for all standard operations. For example, viewing the pantry and adding an item.
- Data repository and distribution requirements: All persistent data will be stored in a MySQL relational database. Which it will be hosted locally during development.
- Memory and other capacity limitations: Must be able to store 500 items per user at a time.
- Other means of addressing quality goals: Interface should be easy and intuitive to use for nontechnical users. The system have to tolerate invalid input, and fail securely, without breaking anything.
- Availability or volatility of resources: The system availability will be limited during development depending on personal computers, local hostings and internet connectivity.
- Interface/protocol requirements: Must report errors to the user including, network errors, authorization failures and validations failure.
- Security requirements (or other such regulations): Password must be securely hashed using Argon2. And user input will be sanitized and validated in the server side to prevent injections.
- Network communications: All frontend and backend communication will use HTTP protocol using ngrok tunneling.
- Standards compliance: The system follows established web development standards. Frontend components are built using HTML5, CSS3, and JavaScript (ES6). Backend code is written in Python following PEP8 style guidelines.
- End-user environment: The system is intended to be accessed through modern web browsers such as Chrome, Firefox, Safari, and Edge. It will be primarily used on desktop and laptop devices during early testing stages. No installation is required by the end-user.
- Verification and validation requirements (testing): The system must be testable during development and QA to ensure all core functionality works as intended. This includes manual black-box testing of user-facing features such as login, pantry item management, and shopping list creation.

3. System Architecture

User management: manages secure access/verification across users, it handles the job of signin, login, hashing, password storage, 2 factor authentication, resetting passwords/usernames, changing passwords/usernames, deleting accounts, and session

management.

Pantry management: allows users to add, update and delete items, it has to store variables like name, quantities, expiration dates and its link to a certain user. Items can be sorted depending on any of the variables. As well as dealing with input validation from user input.

Expiration date reminders: job to identify all items with upcoming expiration dates and notify the users though many options provided to the user such as email notification or push notification. All close to expiring items will be highlighted to the user

Shopping list management: users can generate shipping lists based on either out-of-stock items or expiration data. It can also label things as purchased which would remove it from the shopping list.

Security management: input validation input needs to be sanitized before processing,
sessions when a user login they are given a JWT key through the browser that can allows them access this key for security reasons like changing passwords/email needs to be revoked by the server, lastly Role-based access control the pantry and shopping list needs to be private and secure to the user who created them.

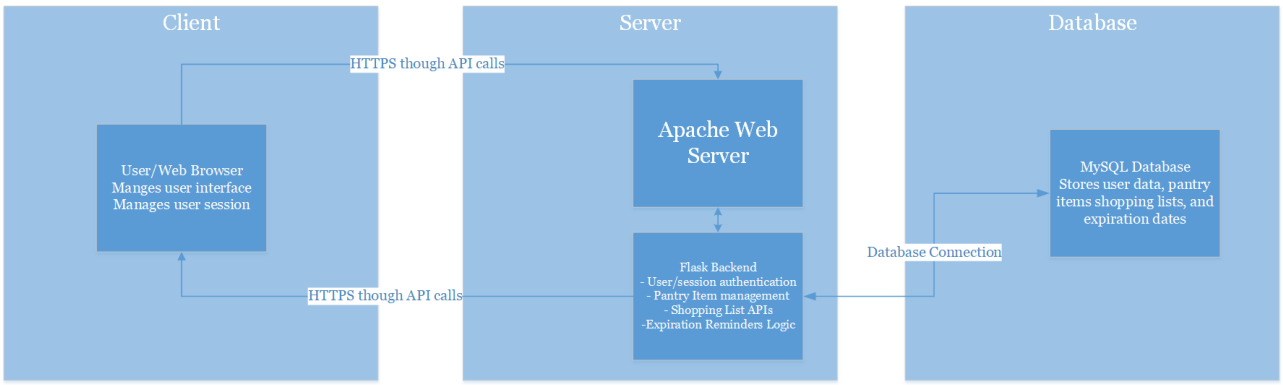
Components/Communications:

Database: It needs to be able to store all of the system data, which includes user information, such as passwords, usernames, emails and any user settings, all items in the pantry and shopping list, and it needs to be able to store the session keys given to the user. Application used is MySQL. It needs to connect directly to the application server.

Application/web server: It needs to do all the logic of the system, which includes receiving api requests from the web browser, communicate to the database, send push notifications through an email API, and lastly respond to any requests from the web browser through APIs (GET, POST, PUT, DELETE).

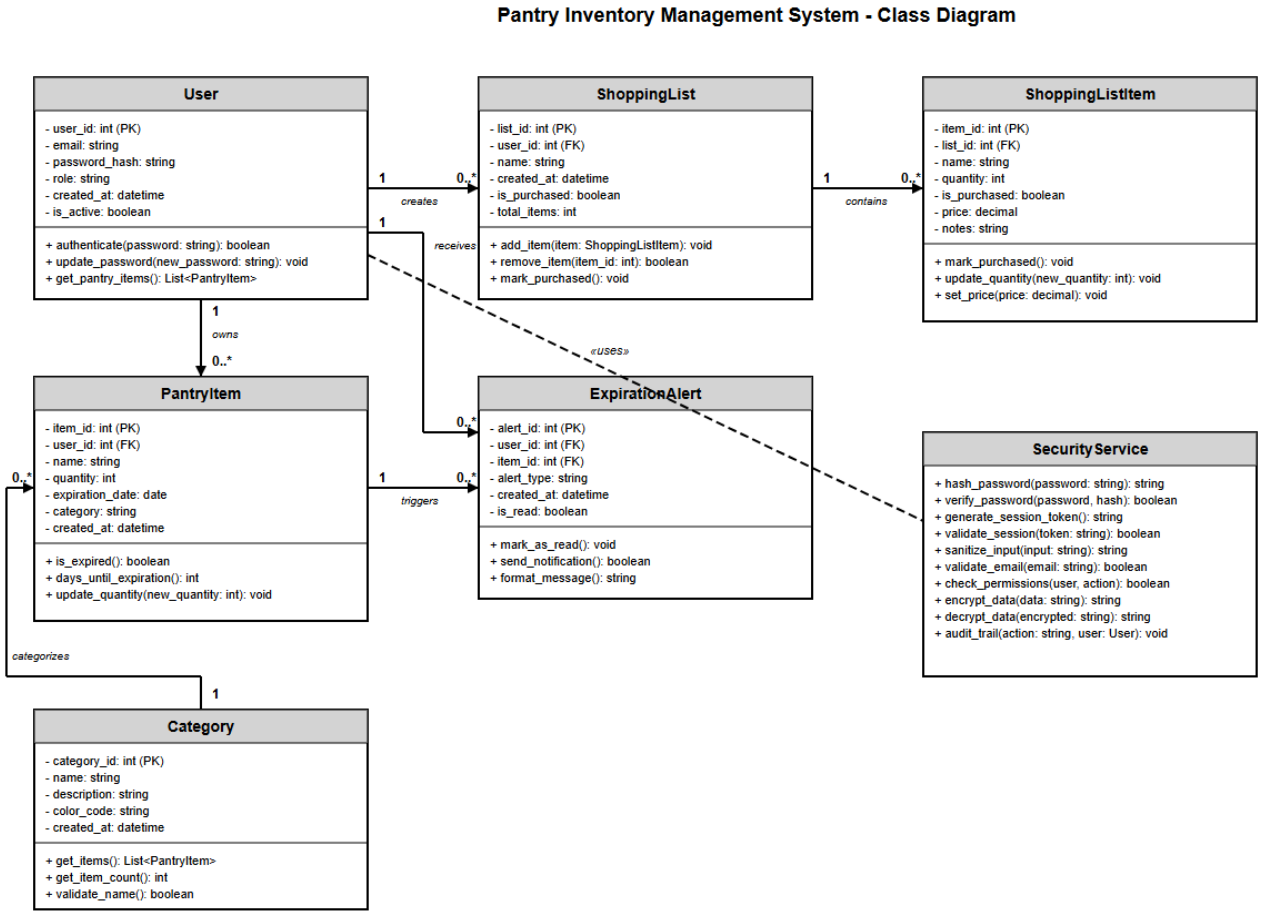
Web Browser: it needs to load the HTML, CSS, and Javascript. It needs to communicate with the application server to execute the functionality. It deals with session management and any request needed to query or modify data from the server

Client/ User interface: it provides the user interface for the user to manipulate and gain information from. It will display user login/signup, pantry items, shopping lists and allow for input.



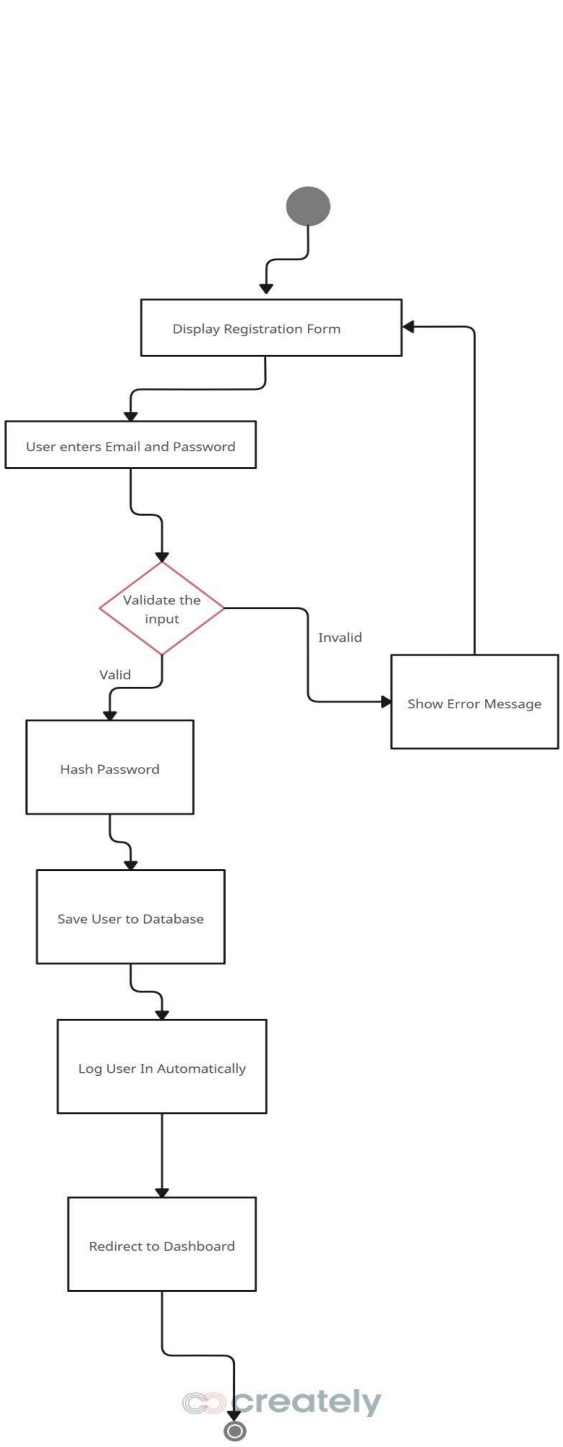
4. Detailed System Design

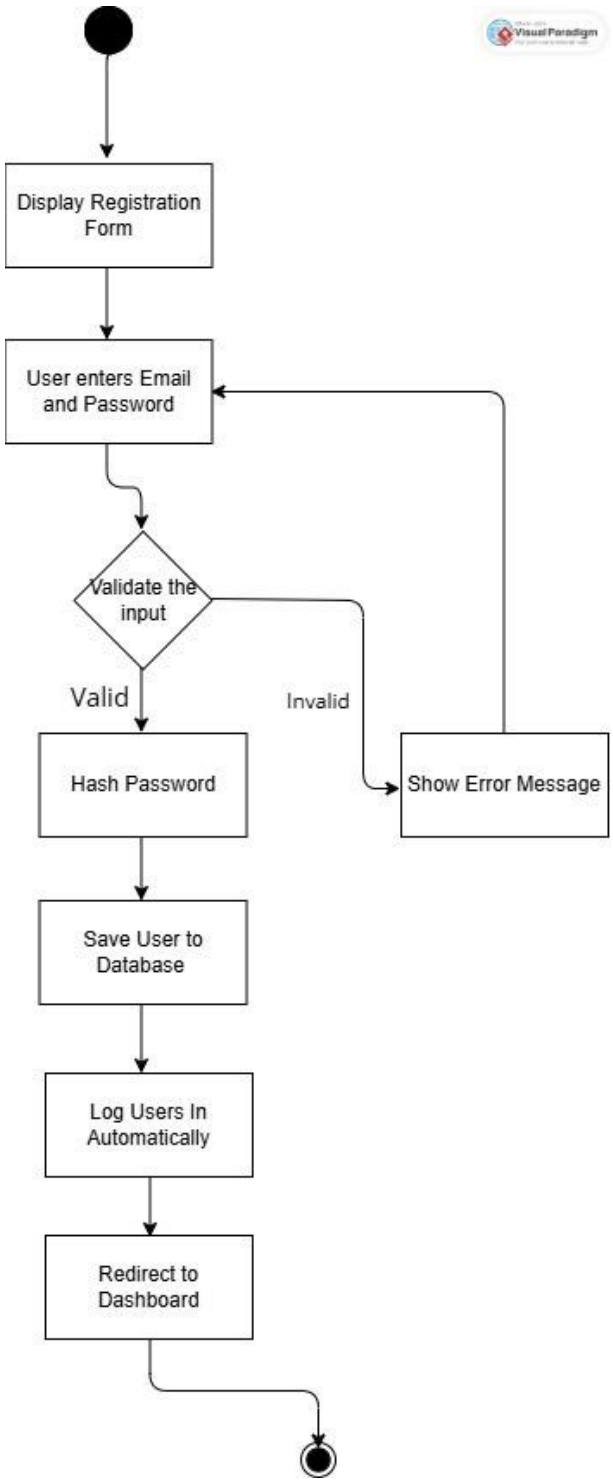
4.1 Class Diagram

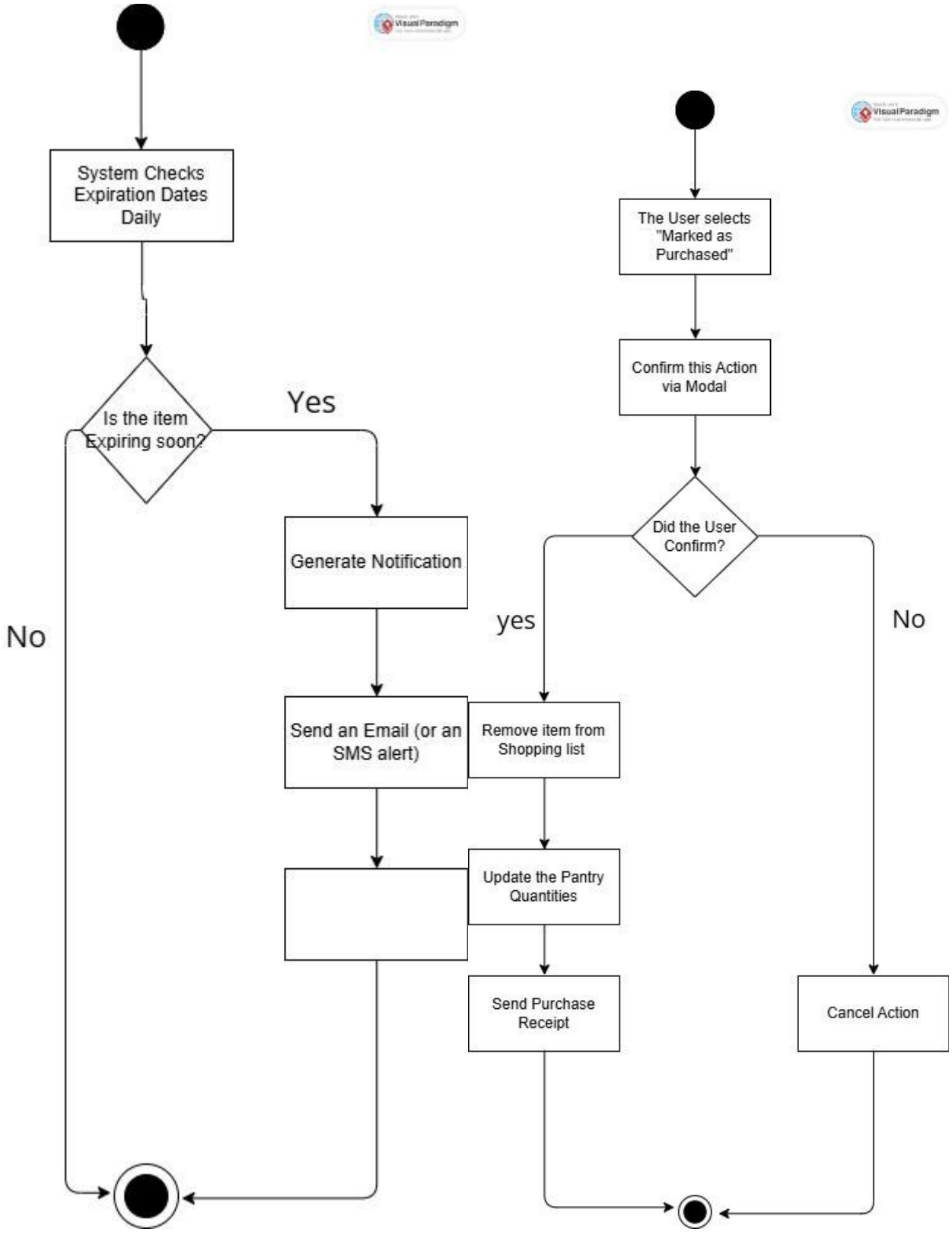


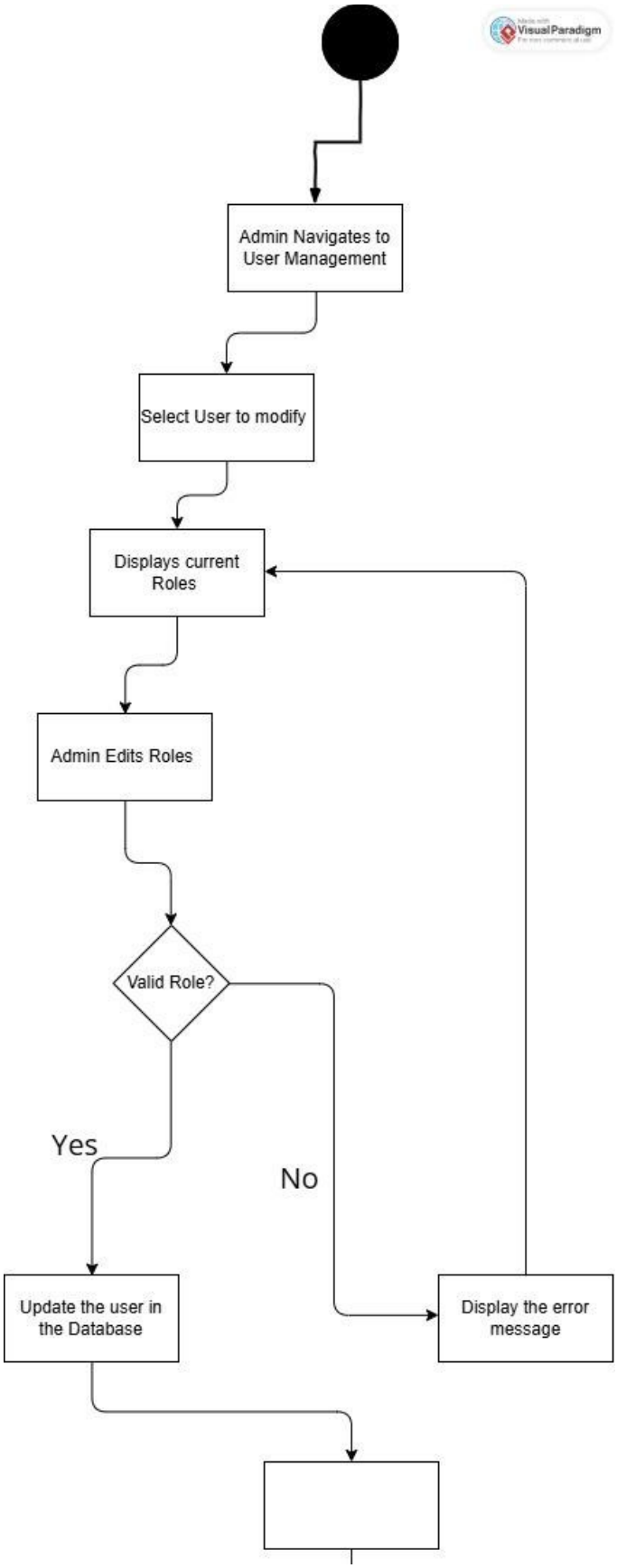
4.2 Activity Diagram

These diagrams go in order from User Registration, Add Item to Pantry, Expiration Alert System, Shopping list Purchase, and Admin Role Management.

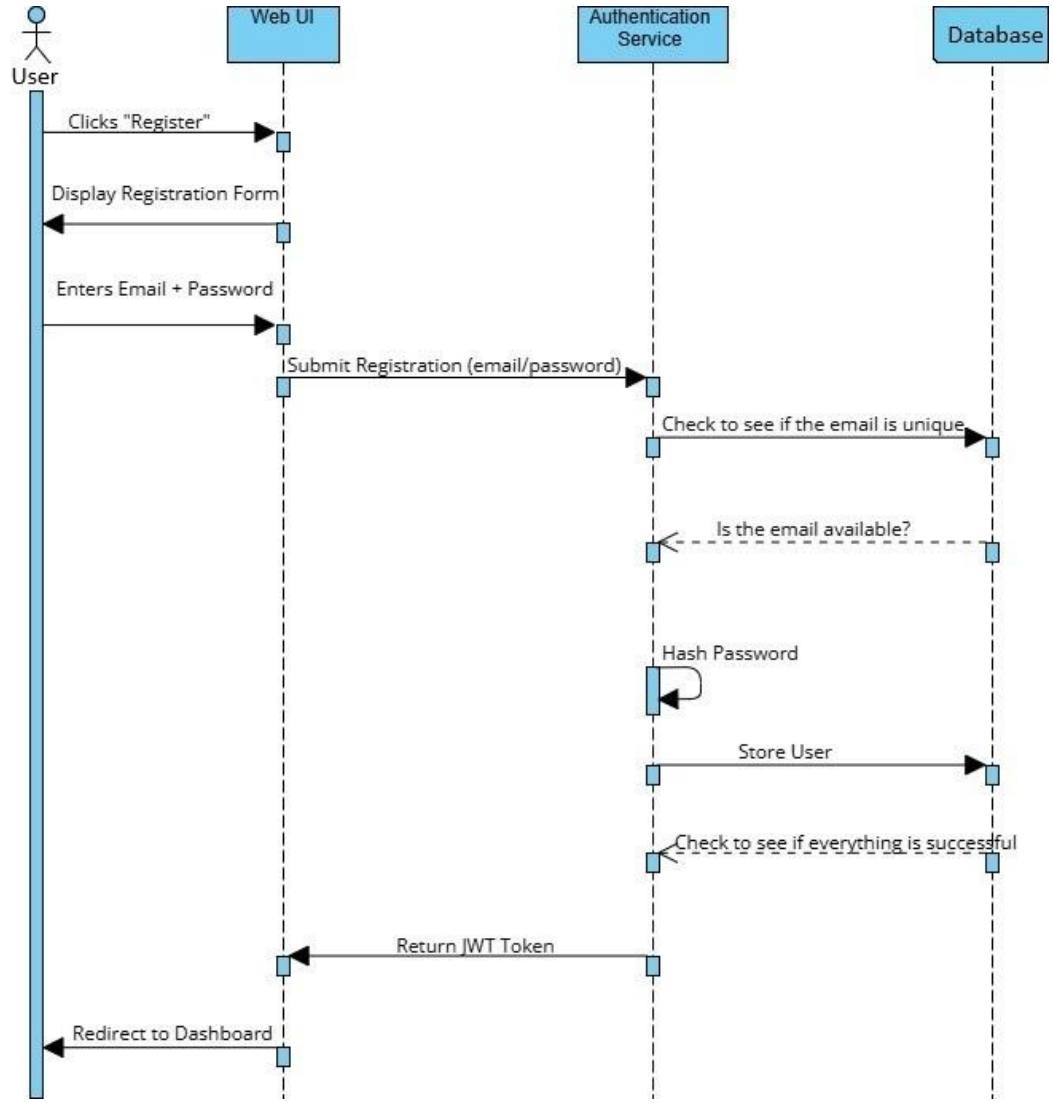


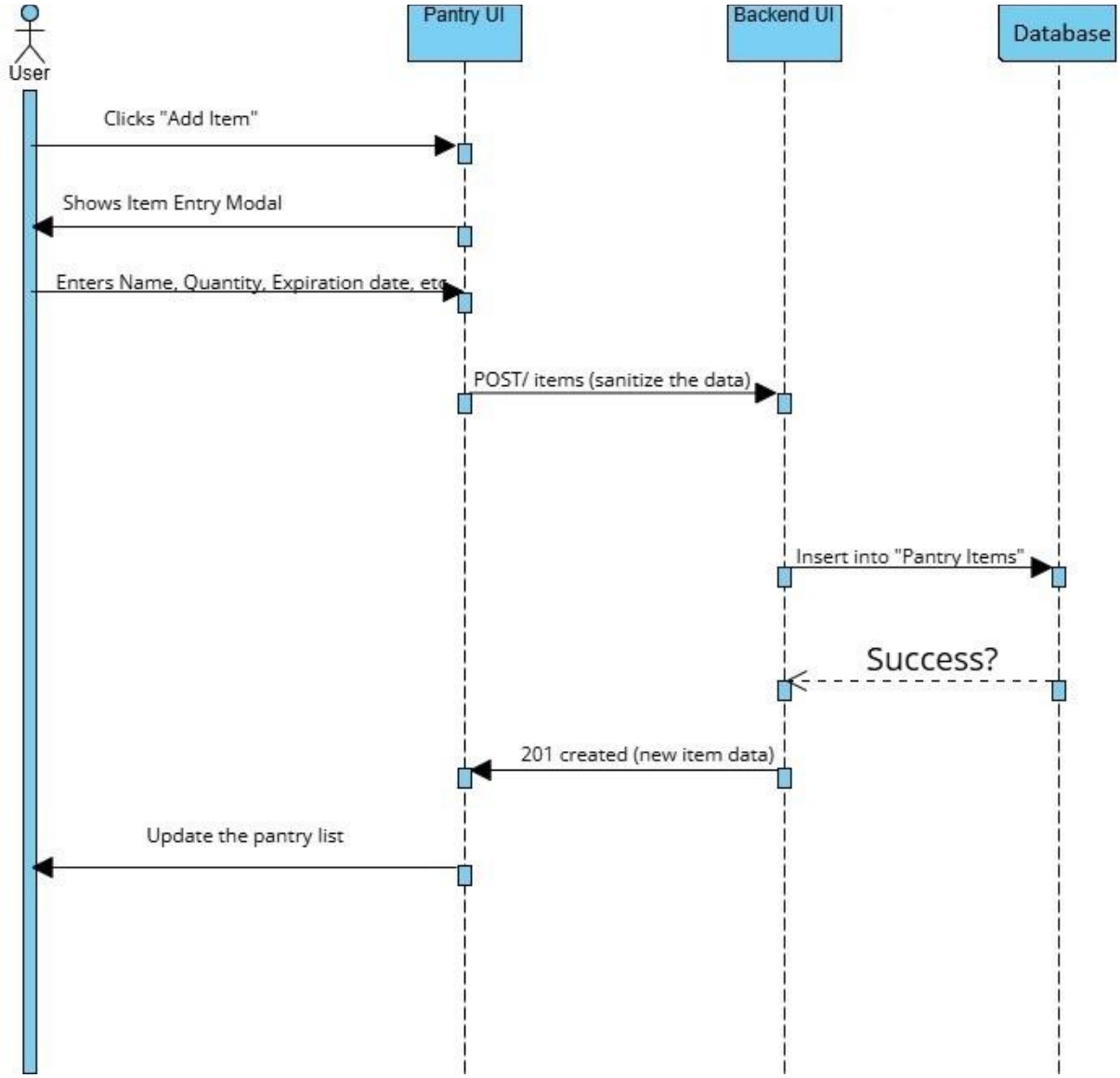


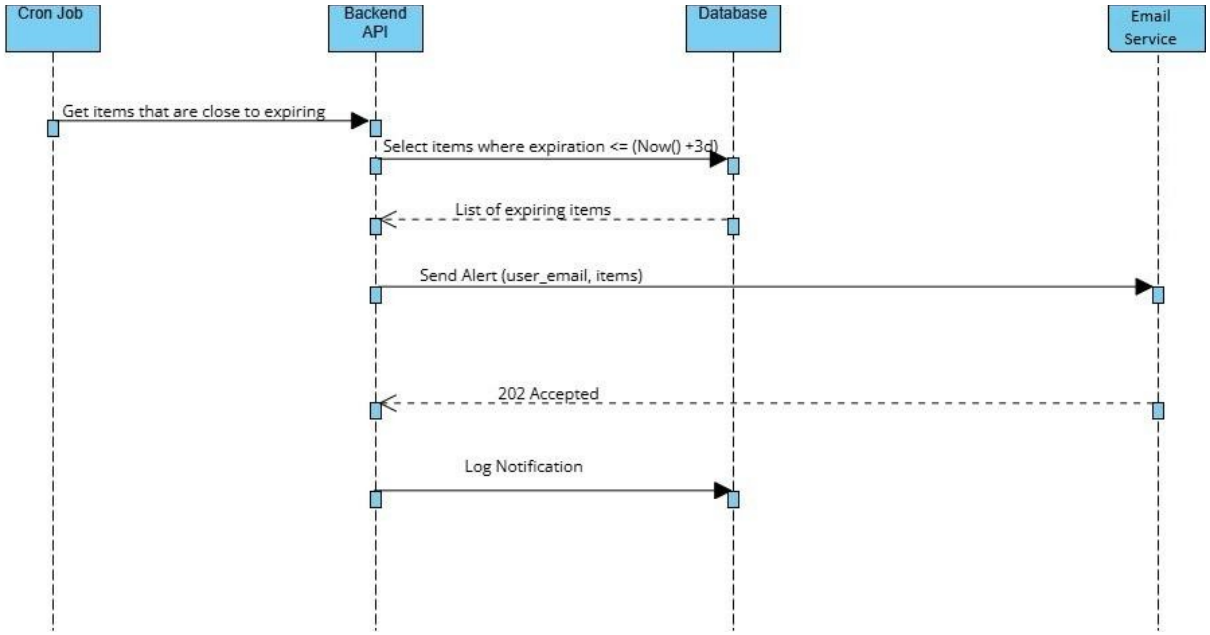

System Checks Expiration Dates Daily
Is the item Expiring soon?
Yes
No
Generate Notification
Send an Email (or an SMS alert)
The User selects "Marked as Purchased"
Confirm this Action via Modal
Did the User Confirm?
yes
No
Remove item from Shopping list
Update the Pantry Quantities
Send Purchase Receipt
Cancel Action

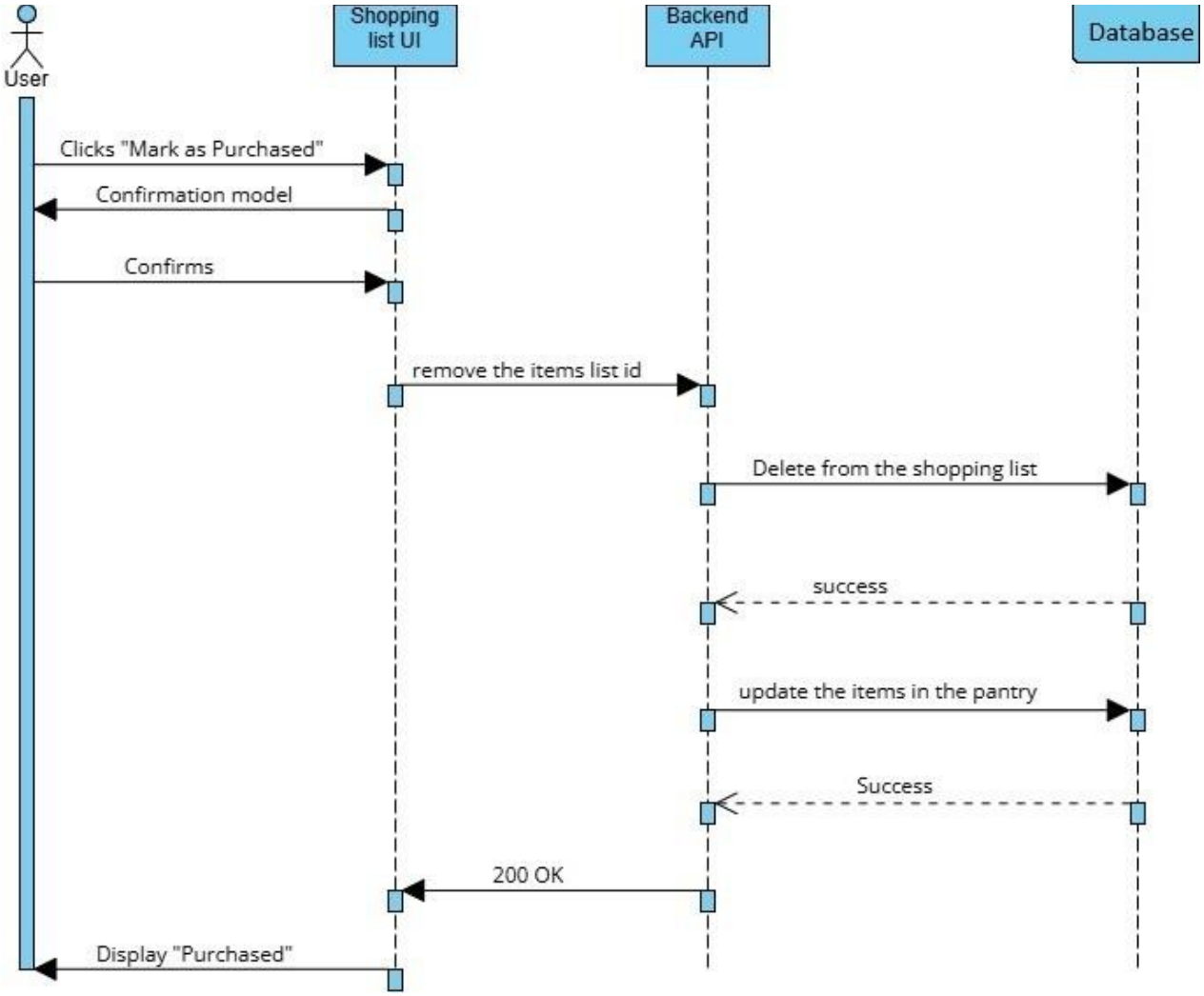

Admin Navigates to User Management
Select User to modify
Displays current Roles
Admin Edits Roles
Valid Role?
Yes
No
Update the user in the Database
Display the error message

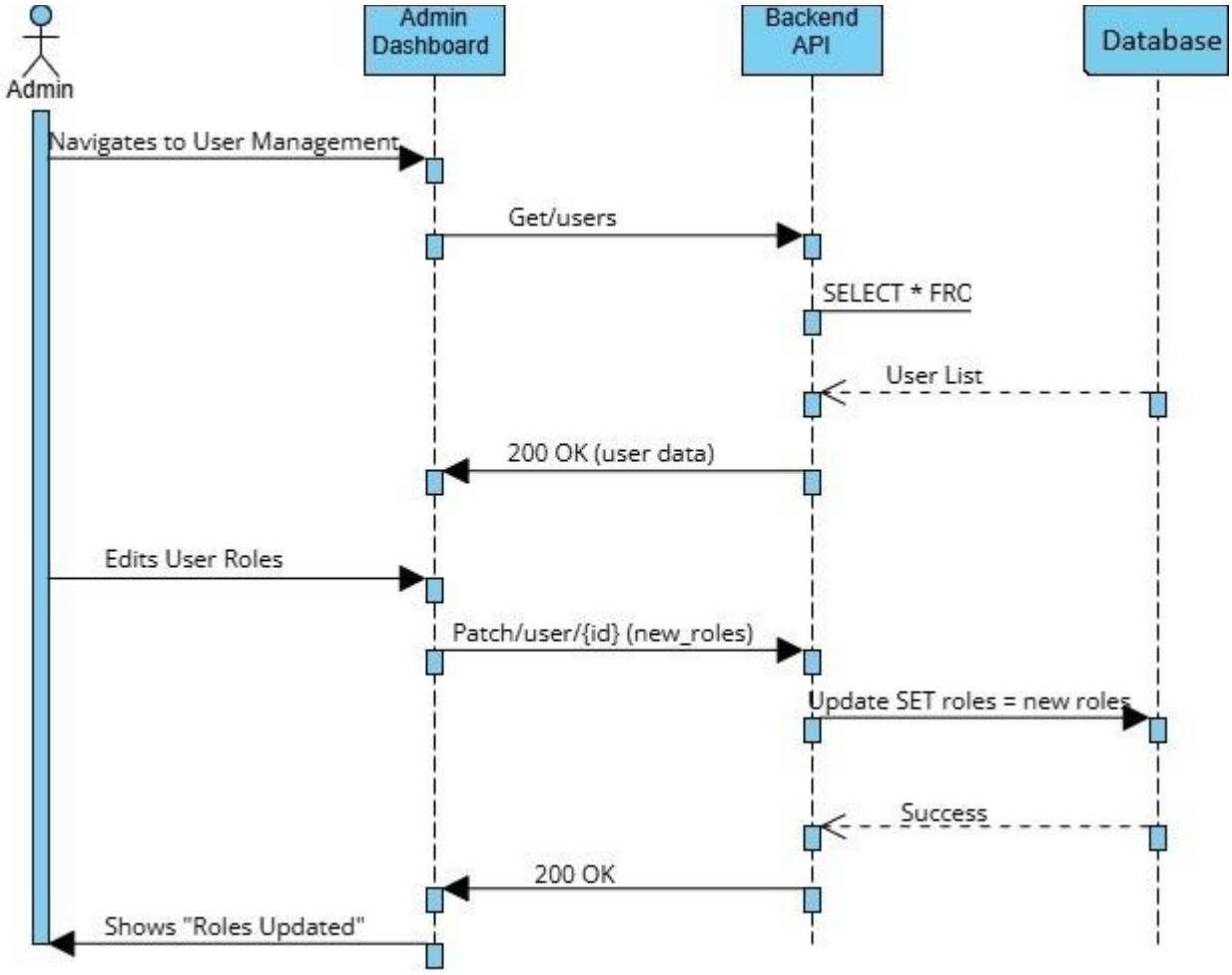
4.3 Sequence Diagram



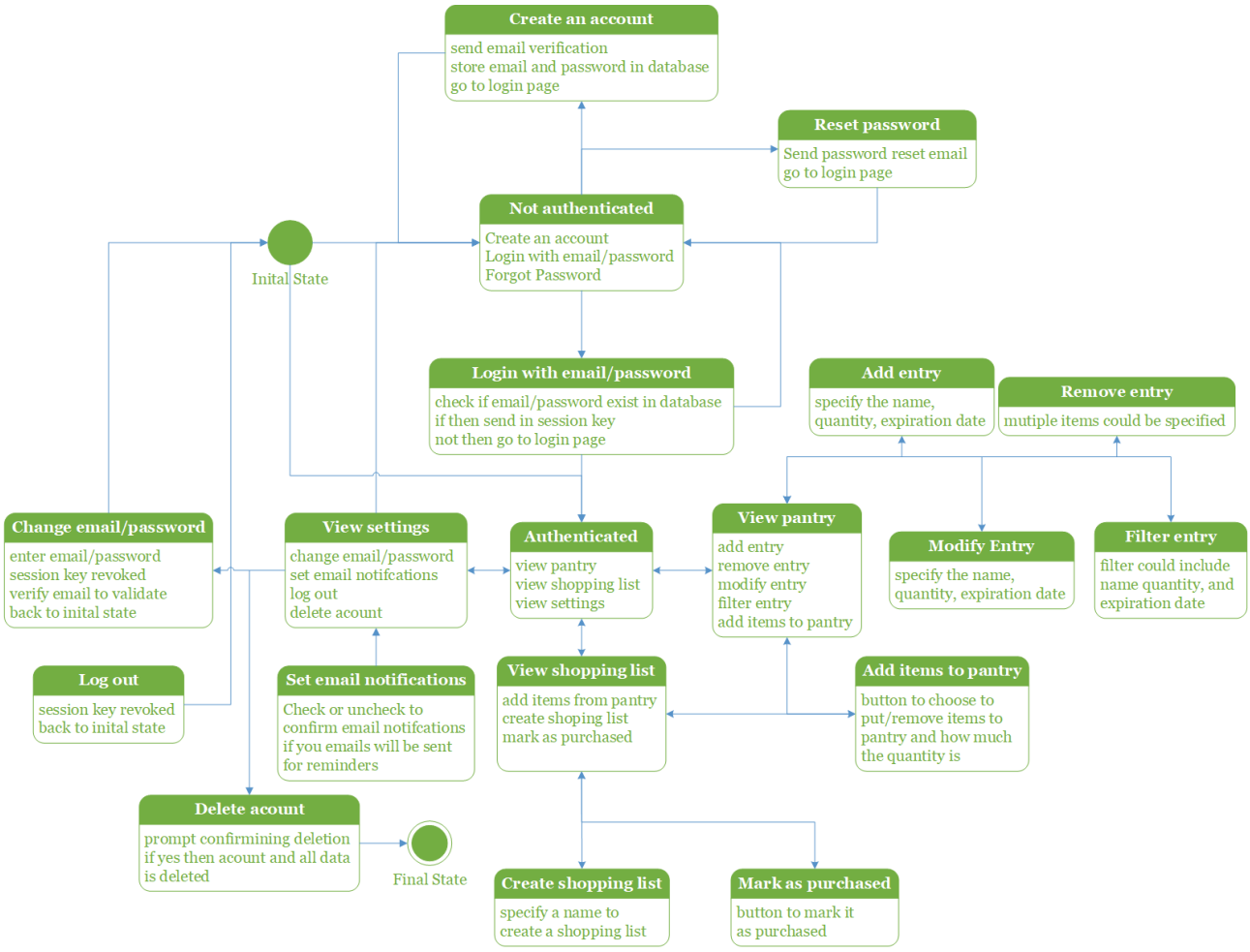

User
Pantry UI
Backend UI
Database
Clicks "Add Item"
Shows Item Entry Modal
Enters Name, Quantity, Expiration date, etc
POST/ items (sanitize the data)
Insert into "Pantry Items"
Success?
201 created (new item data)
Update the pantry list


Cron Job
Backend API
Database
Email Service
Get items that are close to expiring
Select items where expiration <= (Now() +3d)
List of expiring items
Send Alert (user_email, items)
202 Accepted
Log Notification


User
Shopping list UI
Backend API
Database
Clicks "Mark as Purchased"
Confirmation model
Confirms
remove the items list id
Delete from the shopping list
success
update the items in the pantry
Success
200 OK
Display "Purchased"


Admin
Admin Dashboard
Backend API
Database
Navigates to User Management
Get/users
SELECT * FRC
User List
200 OK (user data)
Edits User Roles
Patch/user/{id} (new_roles)
Update SET roles = new roles
Success
200 OK
Shows "Roles Updated"

4.4 State Transition Diagram



4.5 Algorithms for Components/Methods

4.5.1 Expiration Alert System

```
FUNCTION checkExpiringItems():
    today = getCurrentDate()
    expiringItems = database.query("SELECT * FROM pantry_items WHERE
expiration_date <= today + 3 days")

    FOR each item in expiringItems:
        message = "Item " + item.name + " expires on " + item.expiration_date
        database.insert("notifications", user_id=item.user_id,
message=message)
        sendEmail(item.user_email, message)
    END FOR
END FUNCTION
```

4.5.2 User Authentication

```
FUNCTION authenticateUser(email, password):
    user = database.query("SELECT * FROM users WHERE email = email")

    IF user exists:
        hashedPassword = argon2.hash(password)
        IF hashedPassword matches user.password_hash:
            token = generateJWT(user.id)
            RETURN token
        ELSE:
            RETURN "Invalid password"
        END IF
    ELSE:
        RETURN "User not found"
    END IF
END FUNCTION
```

4.5.3 Auto-Generate Shopping List

```
FUNCTION generateShoppingList(user_id):
    lowItems = database.query("SELECT * FROM pantry_items WHERE user_id =
user_id AND quantity < 2")
    expiredItems = database.query("SELECT * FROM pantry_items WHERE user_id =
user_id AND expiration_date < today")

    FOR each item in lowItems + expiredItems:
        IF item not already in shopping list:
            database.insert("shopping_list_items", list_id=user_shopping_list,
item_name=item.name, quantity=1)
        END IF
    END FOR
END FUNCTION
```

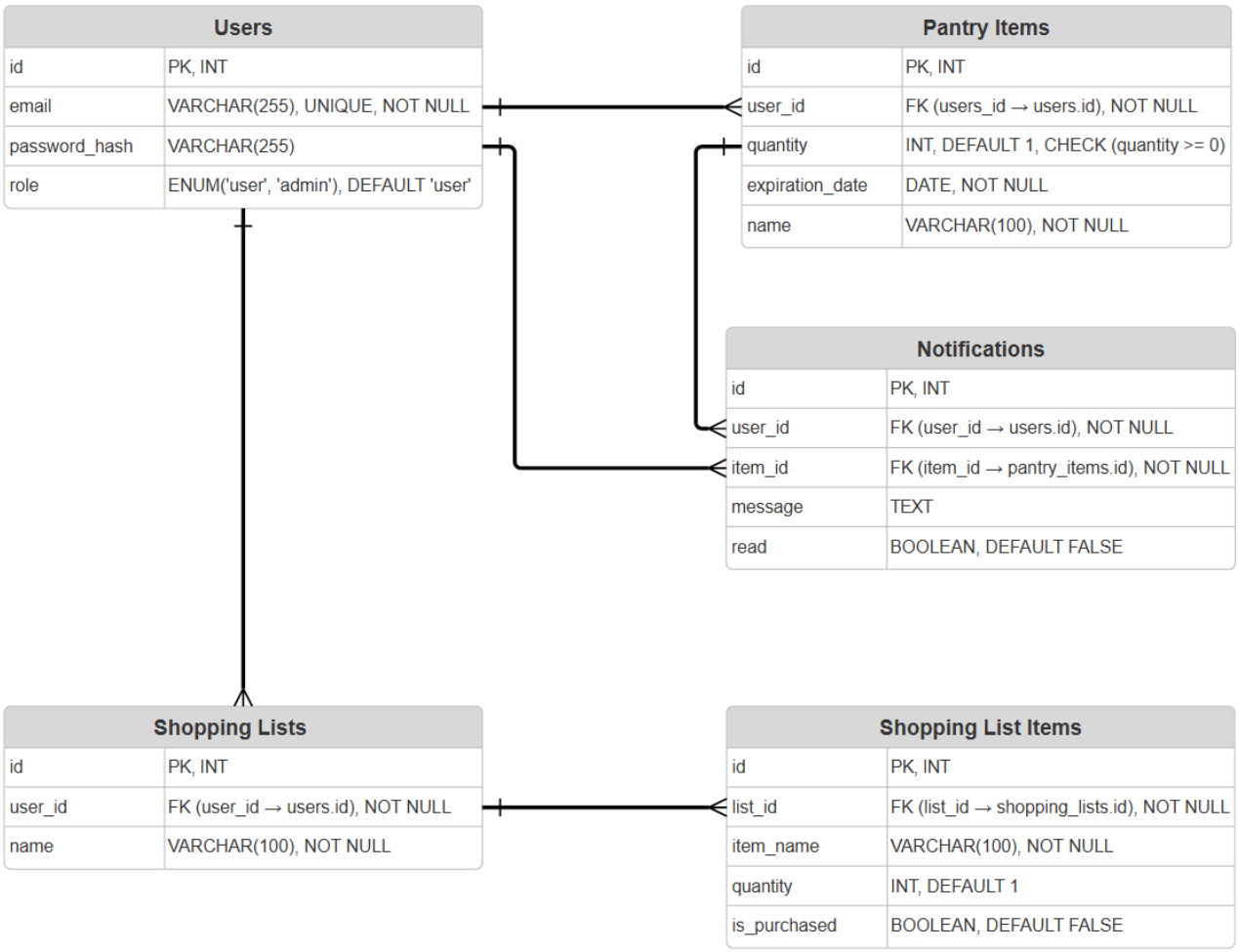
4.5.4 Shopping List Purchase Processing

```
FUNCTION markAsPurchased(shopping_item_id):
    item = database.query("SELECT * FROM shopping_list_items WHERE id =
shopping_item_id")

    database.update("shopping_list_items SET is_purchased = TRUE WHERE id =
shopping_item_id")

    existingPantryItem = database.query("SELECT * FROM pantry_items WHERE name
= item.item_name")
    IF existingPantryItem exists:
        database.update("pantry_items SET quantity = quantity + item.quantity
WHERE id = existingPantryItem.id")
    ELSE:
        database.insert("pantry_items", name=item.item_name,
quantity=item.quantity, user_id=user_id)
    END IF
END FUNCTION
```

4.6 Database Design



Users

- **Purpose:** Stores information about the registered users of the application.
- **Fields:**
  - `id`
    - **Data Type:** `INT`
    - **Constraints:** `PK`, `NOT NULL`
    - **Description:** Unique identifier for each user.
  - `email`
    - **Data Type:** `VARCHAR(255)`
    - **Constraints:** `UNIQUE`, `NOT NULL`
    - **Description:** The user's email address, used for login and as a unique identifier.
  - `password_hash`

        - **Data Type:** VARCHAR(255)
        - **Constraints:** NOT NULL
        - **Description:** Stores the hashed version of the user's password for security.
    - role
        - **Data Type:** ENUM('user', 'admin')
        - **Constraints:** NOT NULL, DEFAULT 'user'
        - **Description:** Defines the user's role in the system (e.g., standard user, administrator).
- **Primary Key (PK):** id
- **Alternative Key (AK):** email (Ensures each user has a unique email address).
- **Relationships:**
    - One-to-Many with Pantry Items (A user can have many pantry items).
    - One-to-Many with Shopping Lists (A user can have many shopping lists).
    - One-to-Many with Notifications (A user can receive many notifications).
- **Integrity Rules/Constraints:**
    - email must be unique across all users.
    - password_hash must always be present.
    - role must be either 'user' or 'admin'.

Pantry Items

- **Purpose:** Stores information about food items a user currently has in their pantry.
- **Fields:**
    - id
        - **Data Type:** INT
        - **Constraints:** PK, NOT NULL
        - **Description:** Unique identifier for each pantry item record.
    - user_id
        - **Data Type:** INT
        - **Constraints:** FK (user_id --> users.id), NOT NULL
        - **Description:** Links the pantry item to the owning user.
    - quantity
        - **Data Type:** INT
        - **Constraints:** DEFAULT 1, CHECK (quantity >= 0), NOT NULL
        - **Description:** The number of units of the item in the pantry.
    - expiration_date
        - **Data Type:** DATE
        - **Constraints:** NOT NULL
        - **Description:** The expiration date of the pantry item.
    - name
        - **Data Type:** VARCHAR(100)

        - **Constraints:** NOT NULL
        - **Description:** The name or description of the pantry item (e.g., "Milk", "Canned Beans").
- **Primary Key (PK):** id
- **Foreign Key (FK):**
    - user_id references Users.id
- **Integrity Rules/Constraints:**
    - quantity must be a non-negative integer.
    - expiration_date must always be provided for a pantry item.

Notifications

- **Purpose:** Stores notifications generated by the system for users (e.g., expiring pantry items).
- **Fields:**
    - id
        - **Data Type:** INT
        - **Constraints:** PK, NOT NULL
        - **Description:** Unique identifier for each notification.
    - user_id
        - **Data Type:** INT
        - **Constraints:** FK (user_id --> users.id), NOT NULL
        - **Description:** Links the notification to the recipient user.
    - item_id
        - **Data Type:** INT
        - **Constraints:** FK (item_id --> pantry_items.id), NOT NULL
        - **Description:** Links the notification to a specific Pantry Item that it pertains to (e.g., an item nearing expiration).
    - message
        - **Data Type:** TEXT
        - **Constraints:** NOT NULL
        - **Description:** The content of the notification message.
    - read
        - **Data Type:** BOOLEAN
        - **Constraints:** DEFAULT FALSE, NOT NULL
        - **Description:** Indicates whether the user has marked the notification as read.
- **Primary Key (PK):** id
- **Foreign Keys (FKs):**
    - user_id references Users.id
    - item_id references Pantry Items.id
- **Integrity Rules/Constraints:**

    - `message` must always be present.

Shopping Lists

- **Purpose:** Stores individual shopping lists created by users.
- **Fields:**
    - `id`
        - **Data Type:** `INT`
        - **Constraints:** `PK`, `NOT NULL`
        - **Description:** Unique identifier for each shopping list.
    - `user_id`
        - **Data Type:** `INT`
        - **Constraints:** `FK (user_id --> users.id)`, `NOT NULL`
        - **Description:** Links the shopping list to the owning user.
    - `name`
        - **Data Type:** `VARCHAR(100)`
        - **Constraints:** `NOT NULL`
        - **Description:** The name of the shopping list (e.g., "Weekly Groceries", "Birthday Party").
- **Primary Key (PK):** `id`
- **Foreign Key (FK):**
    - `user_id` references `Users.id`
- **Integrity Rules/Constraints:**
    - `name` must always be present for a shopping list.

Shopping List Items

- **Purpose:** Stores individual items within a specific shopping list.
- **Fields:**
    - `id`
        - **Data Type:** `INT`
        - **Constraints:** `PK`, `NOT NULL`
        - **Description:** Unique identifier for each item on a shopping list.
    - `list_id`
        - **Data Type:** `INT`
        - **Constraints:** `FK (list_id --> shopping_lists.id)`, `NOT NULL`
        - **Description:** Links the shopping list item to its parent shopping list.
    - `item_name`
        - **Data Type:** `VARCHAR(100)`
        - **Constraints:** `NOT NULL`
        - **Description:** The name or description of the item needed for the shopping list (e.g., "Bread", "Eggs").

    - `quantity`
        - **Data Type:** `INT`
        - **Constraints:** `DEFAULT 1`, `NOT NULL`
        - **Description:** The number of units of the item to purchase.
    - `is_purchased`
        - **Data Type:** `BOOLEAN`
        - **Constraints:** `DEFAULT FALSE`, `NOT NULL`
        - **Description:** Flag indicating whether the item has been purchased.
- **Primary Key (PK):** `id`
- **Foreign Key (FK):**
    - `list_id` references `Shopping Lists.id`
- **Integrity Rules/Constraints:**
    - `item_name` must always be present.
    - `quantity` should ideally be a positive integer (though `CHECK` constraint is not explicitly stated in ERD for `quantity >= 1`, it's generally implied for shopping items).

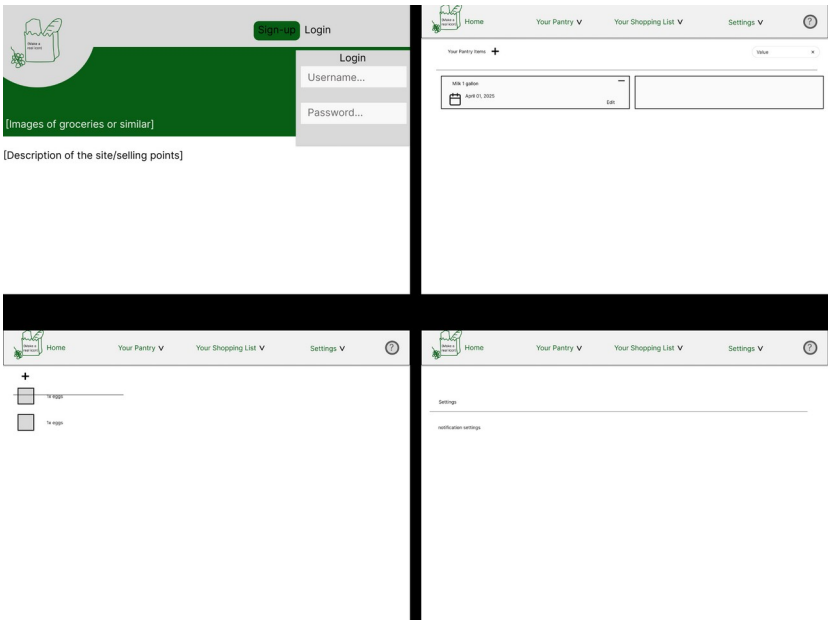
5. User Interface Design

The user interface mockup consists of the 4 most important pages in the system. A home screen contains a brief description of the system and what the user can do with it. Additionally, there are login/sign-up buttons which open respective popups. Logging in will move the user to the next screen that they are immediately trying to access, which is likely the pantry system as the name of our system would suggest.

On the page marked “Your Pantry,” the user can find information about the items currently in their pantry. Next to the calendar icon on each item is the expiration date, as entered by the user when they added this item.

When the user clicks on the overhead button labeled “Your Shopping List,” they are directed to the shopping list page. This page contains information about what the user wants to buy. When the user clicks on the square to check if they have bought it, this item is marked as bought and will stay on this page for some time.

When the user clicks on the settings button at the top bar, they are directed to the settings tab. This tab is specific about notification settings and has different sections for each setting.



Appendix A: Glossary

Activity Diagram: Behavioral diagrams which show the flow of activities and actions for non-trivial methods in the system.

Apache2: Web server software created by Apache.

Class Diagram: UML structural diagram showing the classes and their attributes, methods, and relationships in the system.

Components: Modular parts of the system architecture that handle certain functions

Constraints: Global limitations or restrictions that impact the system's software design.

Cryptographic Primitives: Basic cryptographic algorithms used to ensure near zero probability of password hash collisions.

Database: The MySQL system used to store user information, pantry items, shopping lists, and session data.

Flask: The Python web framework used as the backend of the web application.

Full Backup: A complete backup of the entire database including all data, schema, and objects.

HTTPS: The secure protocol assumed to be used for web communications.

JWT: Token authentication method used for secure session management.

MySQL: A free relational database which uses Structured Query Language (SQL)

ngrok: A tool that creates secure tunnels to localhost, enabling public URL access for development and testing.

Null: Commonly used in programming, this essentially means void of any value or type.

Pseudo Code: Detailed algorithmic descriptions required for non-trivial methods in the system design.

Python 3: The programming language used as the codebase for the application.

RAID Configuration: Redundant Array of Independent Disks setup that provides data redundancy and improved performance through disk mirroring or striping.

Redundancy: The duplication of critical system components or data to increase reliability and prevent data loss in case of hardware failure.

Sequence Diagram: UML diagrams showing interactions between objects for non-trivial scenarios.

State Transition Diagram: Diagram showing the different states and transitions within the system.

Three Tiered Architecture: The architectural pattern used, consisting of client, server, and database layers.

Transaction Log: A database file that records all transactions and database modifications, enabling recovery and maintaining data integrity.

Ubuntu: The Linux-based host operating system for the server environment.

URL: Uniform Resource Locator, the web address used to access the pantry management system.

User Authentication: The process of verifying user identity through login credentials.

User Interface: The web-based interface that provides user interaction and information display.

Web Application: The browser-based application that provides the user interface for the pantry management system.

WSL: Windows Subsystem for Linux, a compatibility layer for running a Linux environment on Windows systems.